

ĐẠI HỌC BÁCH KHOA HÀ NỘI



PROJECT III

Kriky

Trợ lí hỗ trợ giáo viên ra đề và chữa bài

Giảng viên hướng dẫn: Vũ Văn Thiệu

Sinh viên thực hiện: Lê Nguyễn Phước Thành – 20235561

Mục lục

1	Đặt vấn đề	5
1.1	Bối cảnh	5
1.2	Vấn đề cần giải quyết	7
1.3	Mục tiêu sản phẩm	7
1.4	Phạm vi	7
2	Ý tưởng sản phẩm	8
2.1	Luồng tổng thể	8
2.2	Hai giai đoạn làm bài của học sinh	10
2.3	Làm sao phân tích lỗi của học sinh?	10
2.4	Tạo câu hỏi mới dựa trên câu làm sai	10
2.5	Thang điểm	10
2.6	Giáo viên đưa ra quyết định ở đâu?	11
3	Actor và khái niệm nghiệp vụ	12
3.1	Hai actor	12
3.1.1	Teacher	12
3.1.2	Student	12
3.2	Từ điển khái niệm	12
4	Các luồng nghiệp vụ	14
4.1	Workflow 1 — Tạo và duyệt đề	16
4.2	Workflow 2 — Làm bài (giai đoạn 1)	16
4.2.1	Ba mốc thời gian	16
4.3	Workflow 3 — Chấm bài	16
4.4	Workflow 4 — Chữa bài (giai đoạn 2)	17
4.4.1	Giai đoạn 2 đếm giờ như thế nào?	17
4.4.2	Khi nào kết thúc giai đoạn 2	17
4.4.3	Giáo viên làm gì?	18
4.5	Business Workflows	18
5	Use case	19
5.1	Sáu use case	19
5.2	Đặc tả từng use case	21
5.2.1	UC-01 — Tạo đề nháp	21
5.2.2	UC-02 — Review và phát hành đề	21
5.2.3	UC-03 — Làm và nộp bài	21
5.2.4	UC-04 — Chữa bài	21

5.2.5	UC-05 — Tạo câu biến thể	22
5.2.6	UC-06 — Báo cáo Trợ lý giải thích chưa rõ	22
6	Kiến trúc và mô hình dữ liệu	23
6.1	Năm thành phần, và cái gì làm nên một thành phần	23
6.2	Ranh giới quyền quyết định	26
6.3	Ranh giới chìa khoá: ai chạm được kho nào	26
6.4	Các thành phần nói chuyện với nhau thế nào	27
6.5	Thứ dùng chung, và cái giá đã nhận	27
6.6	Việc gì đi qua hàng đợi, việc gì không	28
6.7	Thành phần nào ngừng thì mất gì	29
6.8	Cách giữ cho ranh giới không mục	30
6.9	Kiến trúc không nói nơi chạy	30
6.10	Mô hình dữ liệu	31
6.10.1	Ba chỗ đáng giải thích	32
6.11	Vì sao chưa vẽ sơ đồ lớp hay sơ đồ thực thể	33
	Tài liệu tham khảo	34

Danh mục hình vẽ

1.1	Domain Context	6
2.1	Tổng thể một vòng	9
4.1	Activity Overview	15
4.2	Business Workflows	18
5.1	Use Case Diagram	20
6.1	System Architecture	25

Danh mục bảng

2.1	Ba mức điểm của một câu hỏi	11
5.1	Sáu use case và actor chính của từng cái	19
6.1	Năm thành phần, đọc theo ba trục	24
6.2	Bốn việc thật sự cần gọi mô hình	28
6.3	Mỗi thành phần ngừng thì mất gì, còn gì	29
6.4	Ranh giới, và thứ bắt được khi nó bị phá	30
6.5	Hai mươi một bảng, theo năm nhóm	31

Chương 1

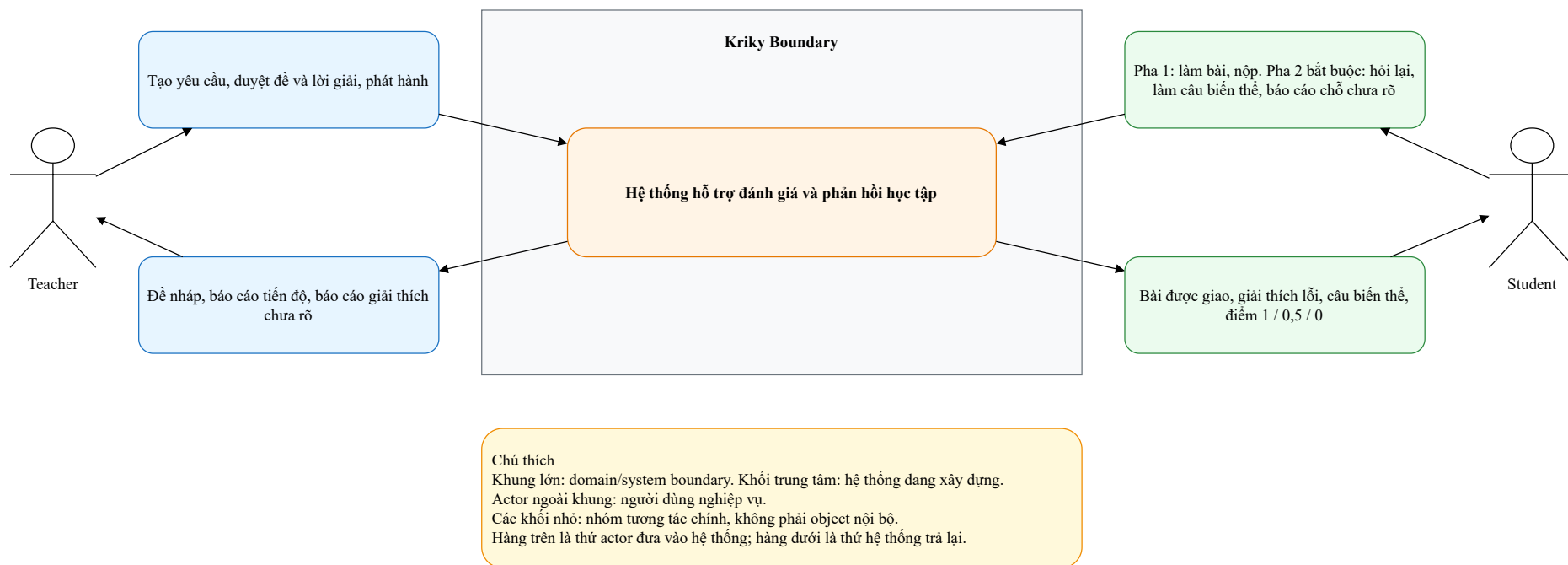
Đặt vấn đề

1.1 Bối cảnh

Ở trường phổ thông, một bài kiểm tra thường kết thúc khi giáo viên trả điểm. Học sinh nhận lại bài, nhìn những câu bị gạch chéo, rồi phần lớn dừng ở đó. Thử thực sự làm thay đổi kết quả học tập là việc hiểu ra mình sai ở đâu và làm lại được, nhưng việc đó hiếm khi xảy ra, vì nó cần một cuộc trao đổi riêng cho từng em về từng câu.

Giáo viên không phải không muốn làm. Vấn đề nằm ở khối lượng công việc. Một lớp bốn mươi học sinh làm một đề mười câu sẽ sinh ra bốn trăm lượt trả lời, và mỗi lượt sai có thể sai vì một lý do khác nhau. Giải thích riêng cho từng em, rồi ra thêm một câu tương tự để kiểm xem em đã sửa được chưa, là công việc tăng theo sĩ số và không có cách nào rút gọn.

Kriky là hệ thống được xây để gánh phần công việc đó, nhưng không lấy đi quyền quyết định của giáo viên. Hình 1.1 cho thấy ranh giới của hệ thống: hai actor ở hai phía, cùng những gì đi vào và đi ra.



Hình 1.1: Domain Context — ranh giới hệ thống, hai actor, và những gì đi qua ranh giới ấy

1.2 Vấn đề cần giải quyết

Phần tốn thời gian nhất của giáo viên không phải là gỡ ra đề bài, mà là năm việc sau. Cả năm đều là công việc chuyên môn:

- Soạn đề bám đúng mục tiêu học tập, thay vì gom câu hỏi cho đủ số lượng.
- Tạo phương án nhiễu có ý nghĩa. Một phương án sai ngẫu nhiên không nói lên điều gì, còn một phương án nhiễu tốt là phương án mà học sinh mắc một lỗi cụ thể sẽ chọn.
- Chấm bài và viết nhận xét cho từng học sinh.
- Phân định học sinh sai vì thiếu kiến thức, vì suy luận hỏng, vì tính nhầm, hay vì hiểu sai một khái niệm.
- Ra câu biến thể của đúng câu học sinh vừa làm sai, để kiểm xem em đã sửa được chỗ sai đó hay chỉ nhớ được đáp án.

Phía học sinh, nhu cầu ngược lại nhưng khớp vào: phản hồi cần đến nhanh, và bài luyện tập cần đúng với điểm yếu của riêng mình. Một phản hồi đúng nhưng đến sau hai tuần thì không còn tác dụng, vì học sinh đã quên mình nghĩ gì lúc làm bài.

1.3 Mục tiêu sản phẩm

Hệ thống hướng tới bốn mục tiêu nghiệp vụ:

1. Hỗ trợ giáo viên tạo đề từ một yêu cầu bằng ngôn ngữ tự nhiên. Một đề có hai nguồn câu hỏi: nội dung sinh mới, và kho câu hỏi đã dùng qua.
2. Phân tích lỗi sai hoặc hiểu nhầm khái niệm của học sinh thông qua cuộc trò chuyện thay vì một câu nhận xét.
3. Tạo các câu hỏi mới từ các câu mà học sinh làm sai ngay trong cùng bài kiểm tra.

1.4 Phạm vi

Hệ thống dựng trọn một vòng nghiệp vụ, thay vì dựng sâu một khâu:

- Giáo viên tạo, xem lại và phát hành đề. Việc duyệt bao gồm cả lời giải và ánh xạ từ phương án nhiễu sang lỗi, không chỉ câu hỏi và đáp án.
- Giai đoạn 1: Học sinh làm bài, nộp bài, và hệ thống chấm bài để biết học sinh đang thiếu kiến thức ở phần nào.
- Giai đoạn 2: Học sinh trò chuyện với trợ lý Kriky để hiểu cách làm bài và tạo sao lại sai. Sau đó hệ thống và tạo câu hỏi mới, học sinh làm lại cho tới khi hiểu được cách làm của dạng bài.

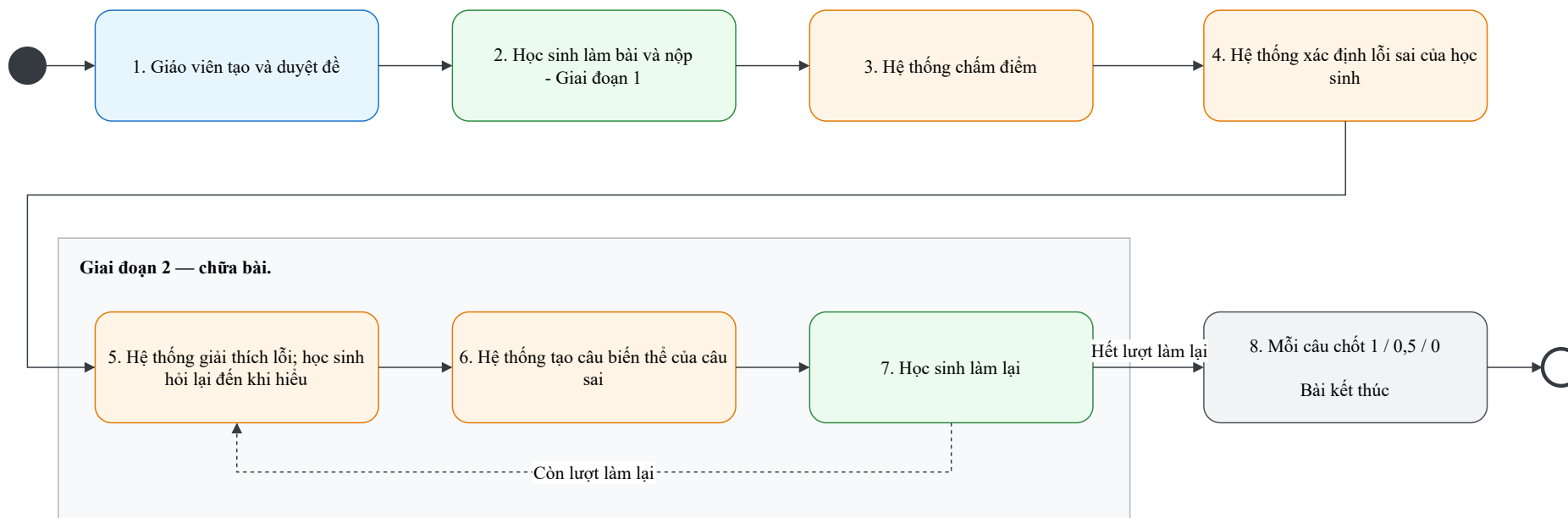
Chương 2

Ý tưởng sản phẩm

2.1 Luồng tổng thể

Ý tưởng lõi không nằm ở việc sinh câu hỏi, mà ở việc khép kín một vòng tám bước:

1. Giáo viên tạo và duyệt đề.
2. Học sinh làm bài.
3. Hệ thống chấm điểm.
4. Hệ thống xác định lỗi sai của học sinh.
5. Hệ thống giải thích lỗi, học sinh hỏi lại đến khi hiểu.
6. Hệ thống tạo câu biến thể của câu sai.
7. Học sinh làm lại.
8. Mỗi câu chốt ở một trong ba mức điểm, và bài kết thúc.



Hình 2.1: Tổng thể một vòng

2.2 Hai giai đoạn làm bài của học sinh

Bước 2, 7 và 8 của hình 2.1.

Một bài kiểm tra có hai giai đoạn: giai đoạn 1 là làm bài và nộp, giai đoạn 2 là chữa những câu đã sai. Nộp bài kết thúc giai đoạn 1 chứ không kết thúc bài. Bài chỉ kết thúc khi giai đoạn 2 xong, hoặc khi hạn của giai đoạn 2 hết.

Kéo theo đó là một hệ quả cần nói rõ: một bài kiểm tra có hai trạng thái hoàn thành khác nhau, *đã nộp* và *đã hoàn thành*, và chúng không thay thế cho nhau được.

2.3 Làm sao phân tích lỗi của học sinh?

Bước 4 của hình 2.1.

Với hình thức trắc nghiệm, việc chấm chỉ là một phép so. Khó khăn nằm ở chẩn đoán lỗi sai của học sinh, bởi cùng chọn một phương án sai có thể do nhiều lỗi khác nhau sinh ra. Nếu hệ thống đoán lỗi rồi đi giải thích một lỗi học sinh không hề mắc thì nó còn tệ hơn im lặng: em vừa mất thời gian, vừa được dạy sai chỗ.

Vì vậy, mỗi phương án sai phải mang theo lỗi mà nó đại diện, và được soạn cùng câu hỏi chứ không suy ra lúc chấm.

Hai luật đi kèm:

- Mỗi câu hỏi phải mang theo lời giải có nhiều hơn một cách làm.
- Số phương án không cố định. Một câu có thể có ba, bốn hay năm lựa chọn tùy từng câu, nhưng chỉ đúng một lựa chọn là đáp án đúng. Mọi phương án còn lại đều là nhiễu, nên đều phải có lỗi gắn kèm.

2.4 Tạo câu hỏi mới dựa trên câu làm sai

Bước 6 và 7 của hình 2.1

Khi học sinh sai câu 3, hệ thống sinh câu 3', gọi là câu biến thể. Thứ cần chữa là một lỗi cụ thể vừa xảy ra, không phải một chủ đề.

Câu biến thể giữ nguyên dạng đề và cách làm, chỉ đổi dữ kiện, con số và cách hỏi. Đó là cách duy nhất kiểm được rằng học sinh đã sửa đúng chỗ mình sai, chứ không phải chỉ nhớ được đáp án.

2.5 Thang điểm

Bước 8 của hình 2.1.

Ba mức thay vì hai, vì hai mức sẽ biến việc chữa bài thành công việc không được có ý nghĩa. Một học sinh sửa được lỗi của mình đã học đúng thứ mà bài kiểm tra muốn dạy, và mức giữa là chỗ duy nhất ghi nhận điều đó. Vì vậy giai đoạn 2 chỉ cộng chứ không bao giờ trừ: điểm tính ngay sau khi nộp là sẵn.

Bảng 2.1: Ba mức điểm của một câu hỏi

Điểm	Nghĩa
1	Đúng ngay ở giai đoạn 1
0,5	Sai ở giai đoạn 1 nhưng chữa được ở giai đoạn 2
0	Sai và không chữa được

2.6 Giáo viên đưa ra quyết định ở đâu?

Bước 1 của hình 2.1.

Với nội dung đề, trợ lý viết còn giáo viên quyết nội dung đó có đến tay học sinh hay không. Giáo viên bắt buộc tham gia ở hai điểm, một ở đầu ra và một ở đầu vào:

1. Trợ lý hỏi lại khi chưa đủ thông tin, thay vì tự đoán (đầu vào).
2. Duyệt đề trước khi phát hành (đầu ra).

Chương 3

Actor và khái niệm nghiệp vụ

3.1 Hai actor

Hệ thống có đúng hai actor nghiệp vụ: **Teacher** và **Student**.

3.1.1 Teacher

Teacher là người đưa ra các quyết định. Teacher có thể:

- Nhập yêu cầu tạo đề.
- Xem và chỉnh sửa đề do trợ lí tạo.
- Duyệt đề trước khi phát hành.
- Phát hành đề cho một hoặc nhiều lớp.
- Xem kết quả làm bài của học sinh.
- Xem báo cáo học sinh gửi về chỗ trợ lí giải thích chưa rõ.

3.1.2 Student

Student là người nhận đề, làm bài và luyện tập. Student có thể:

- Xem bài kiểm tra được giao.
- Chọn đáp án và nộp bài (giai đoạn 1).
- Nghe giải thích lỗi và hỏi lại đến khi hiểu (giai đoạn 2).
- Làm câu biến thể của những câu đã sai (giai đoạn 2).
- Báo cáo chỗ trợ lí giải thích chưa rõ.

3.2 Từ điển khái niệm

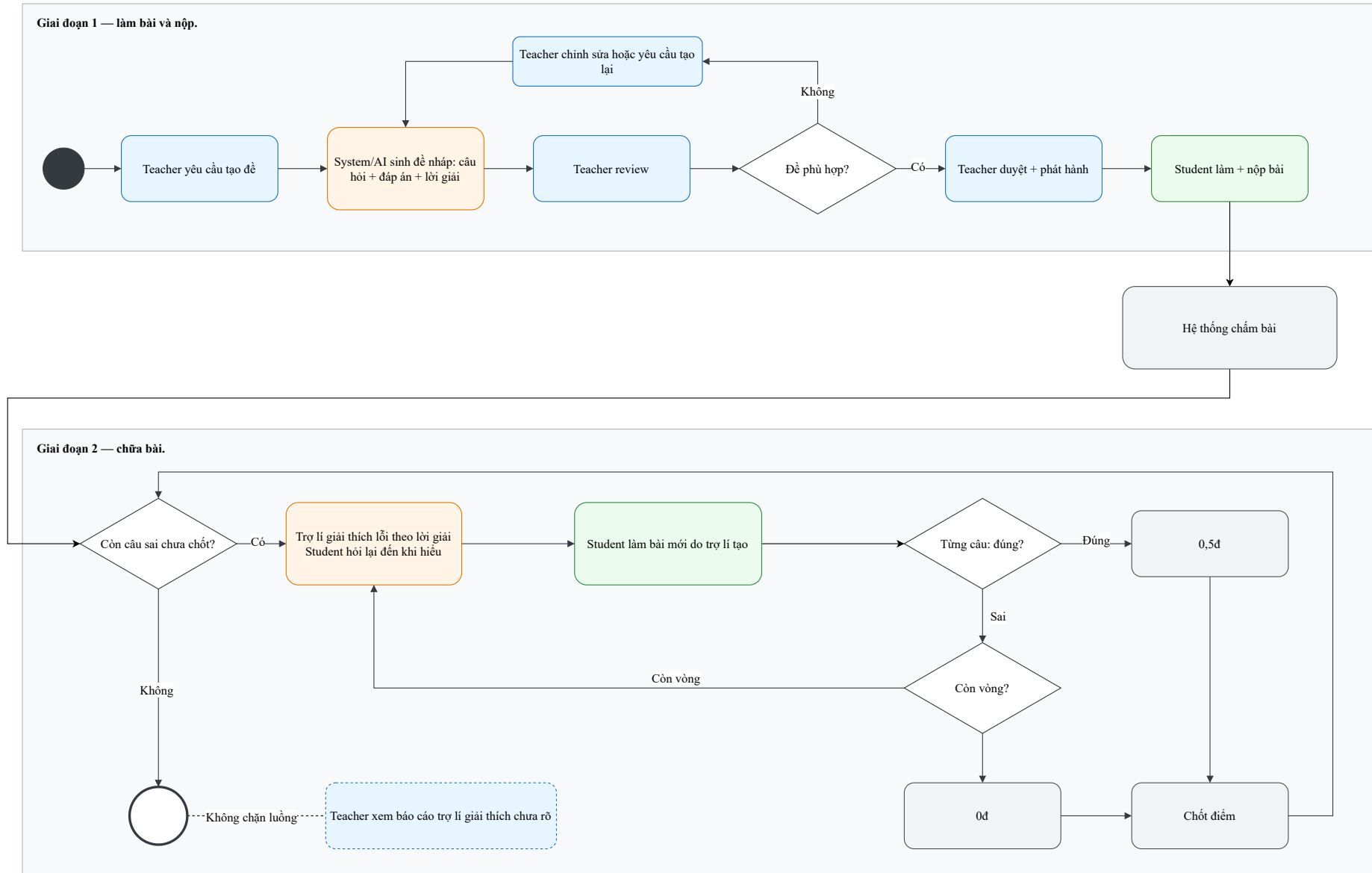
- **Assessment:** Bài kiểm tra, hay bộ câu hỏi được giao cho học sinh.
- **Question:** Câu hỏi trong một assessment.
- **Distractor:** Phương án sai *có chủ đích*, đại diện cho một lỗi hoặc một hiểu nhầm cụ thể. Không phải đáp án nhiễu ngẫu nhiên.
- **Submission:** Bài làm, hay câu trả lời của học sinh.
- **Feedback:** Phản hồi / nhận xét của trợ lí cho những thắc mắc của học sinh.
- **Misconception:** Hiểu nhầm khái niệm, hoặc lỗi sai có tính lặp lại.

- **Mastery:** Mức độ thành thạo của học sinh với một mục tiêu học tập. Khái niệm này vẫn còn trong từ điển nhưng **không còn quyết định gì** — điều kiện dừng của vòng chữa bài là số vòng, không phải mastery.
- **Giai đoạn 1:** Phần làm bài và nộp.
- **Giai đoạn 2:** Phần chữa bài.
- **Lượt:** Một lượt cho phép học sinh làm lại các câu sai của giai đoạn trước / lượt trước.
- **Câu biến thể:** Câu sinh từ *câu học sinh làm sai ở giai đoạn 1*: giữ cấu trúc và lỗi cần kiểm, chỉ đổi dữ kiện.
- **Lời giải:** Cách làm đi kèm mỗi câu hỏi, bắt buộc có nhiều hơn một cách. Trợ lí kriky sẽ dựa vào lời giải để giải thích cho học sinh.
- **Class:** Lớp học có sẵn danh sách học sinh. Giáo viên tạo lớp. Phát hành đề phải chọn một lớp đã tạo từ trước.
- **Question Bank:** Ngân hàng đề.
- **Document:** Tài liệu PDF của giáo viên, nằm trong kho lưu trữ riêng của từng tài khoản. Cung cấp kiến thức và giới hạn phạm vi ra đề.
- **Scope:** Phạm vi ra đề, chương và khoảng trang trong một *Document*.

Chương 4

Các luồng nghiệp vụ

Nghiệp vụ của hệ thống gồm bốn luồng. Hình 4.1 và hình 2.1 mô tả cùng một luồng, nhưng ở hai mức chi tiết khác nhau.



Hình 4.1: Activity Overview — toàn bộ vòng, hai giai đoạn nằm trong hai khung riêng

4.1 Workflow 1 — Tạo và duyệt đề

Mục tiêu. Giúp giáo viên tạo được đề phù hợp với mục tiêu học tập, nhưng giáo viên vẫn là người duyệt cuối cùng trước khi phát hành.

Luồng chính.

1. Teacher nhập yêu cầu tạo đề.
2. Teacher cung cấp ràng buộc: chủ đề, độ khó, số lượng câu hỏi, loại bài kiểm tra, thời gian làm bài.
3. System/AI phân tích yêu cầu.
4. System/AI tạo đề nháp gồm câu hỏi, phương án trả lời, đáp án đúng, lời giải và metadata cơ bản.
5. Teacher xem đề nháp.
6. Teacher chỉnh sửa thủ công, hoặc yêu cầu tạo lại một phần.
7. Teacher duyệt đề.
8. Teacher phát hành đề cho Student.

Điểm kiểm soát. Teacher phải duyệt đề trước khi đề được phát hành. Đề kiểm tra ảnh hưởng trực tiếp đến việc đánh giá học sinh, nên trợ lí không được tự phát hành.

4.2 Workflow 2 — Làm bài (giai đoạn 1)

Mục tiêu. Cho phép học sinh làm bài và ghi nhận bài nộp để hệ thống đánh giá.

Luồng chính.

1. Student nhận assessment đã được Teacher phát hành.
2. Student vào làm, *tới hết* giờ đóng.
3. Student trả lời câu hỏi trắc nghiệm.
4. Student nộp bài.
5. Hệ thống ghi nhận submission để đánh giá.

4.2.1 Ba mốc thời gian

Giai đoạn 1 có ba tham số thời gian độc lập: thời gian làm bài, giờ mở và giờ đóng.

Luật là: giờ đóng chặn việc *vào làm*, không chặn việc nộp. Học sinh vào tới hết giờ đóng vẫn được. Ai đã vào rồi thì không bị dừng giữa chừng, và được đủ thời gian làm bài kể cả khi tràn qua giờ đóng. Ví dụ, đóng lúc 18:00 với thời gian làm bài 15 phút thì bài cuối có thể nộp lúc 18:15.

4.3 Workflow 3 — Chấm bài

Mục tiêu. Biết được những câu mà học sinh làm sai, từ đó trả lời các thắc mắc của học sinh thông qua cuộc trò chuyện và tạo ra các câu hỏi biến thể để kiểm tra độ hiểu bài.

Luồng chính.

1. Học sinh nộp bài.
2. Hệ thống **so** phương án đã chọn với đáp án đúng của câu hỏi đã duyệt. Đây là một phép so, không phải một phán đoán — nên không có gì để mà không chắc.
3. Điểm giai đoạn 1 chốt ngay và học sinh thấy ngay.

Hai điều kiện phải có sẵn từ lúc duyệt đề.

- Mỗi phương án nhiều phải đại diện cho một lỗi cụ thể, soạn cùng lúc với câu hỏi. Phương án nhiều mà không không đại diện cho một lỗi cụ thể thì không giúp học sinh tiến bộ.
- Mỗi câu hỏi phải kèm lời giải. Việc chấm không dùng tới nó, nhưng phương án nhiều phải được tạo dựa trên lời giải thì mới có ý nghĩa và trợ lý phải dựa vào lời giải để giải thích cho học sinh chứ không tự nghĩ ra cách giải mới. Chính vì thế lời giải phải tồn tại từ lúc duyệt đề chứ không sinh ra lúc học sinh hỏi.

4.4 Workflow 4 — Chữa bài (giai đoạn 2)

Mục tiêu. Bài kiểm tra chỉ kết thúc khi học sinh hiểu bài và sửa được lỗi họ mắc phải.

Luồng chính.

1. Học sinh trò chuyện với trợ lý Kriky về lỗi sai, chỗ nào chưa hiểu, cách giải như thế nào.
2. Kriky dựa vào *lời giải đã soạn kèm câu hỏi*. Học sinh hỏi lại đến khi hiểu. Phần này **không tính giờ**.
3. Trong lúc đó, hệ thống sinh câu biến thể của câu mà học sinh làm sai.
4. Học sinh bấm nút làm bài mới. Đồng hồ của lượt bắt đầu chạy.
5. Học sinh làm các câu mới trong lượt.
6. Câu nào đúng thì chốt **0,5** điểm; câu nào còn sai thì sang vòng tiếp.
7. Mỗi câu tối đa **ba vòng**. Hết ba vòng mà vẫn sai thì chốt **0** điểm.
8. Bài kết thúc khi mọi câu đã chốt, hoặc khi hết hạn giai đoạn 2.

4.4.1 Giai đoạn 2 đếm giờ như thế nào?

Giai đoạn 2 nhận hai tham số: số phút mỗi câu, và hạn kết thúc giai đoạn 2.

Thời lượng của một lượt bằng số phút mỗi câu nhân với số câu còn sai trong lượt đó, nên sai hai câu với thời lượng làm lại 5 phút mỗi câu thì lượt ấy dài 10 phút.

Phần giải thích không tính giờ. Đồng hồ chỉ bắt đầu khi học sinh tự bấm nút làm bài mới.

4.4.2 Khi nào kết thúc giai đoạn 2

Hạn kết thúc giai đoạn 2 là một mốc tuyệt đối do giáo viên đặt, chung cho cả lớp, và dài hơn hẳn hạn của giai đoạn 1, chẳng hạn là cuối ngày.

Hết hạn thì dừng, kể cả khi học sinh đang làm dở một lượt, và câu chưa xong nhận 0 điểm.

Đây là chỗ giai đoạn 2 làm ngược với giai đoạn 1. Ở giai đoạn 1, học sinh đã vào thì không bị dừng giữa chừng; ở giai đoạn 2, hết hạn là dừng. Hai luật nghịch nhau một cách có chủ ý: thời gian ở giai đoạn 1 là một lượng hệ thống cấp cho từng người kể từ lúc em vào, còn hạn

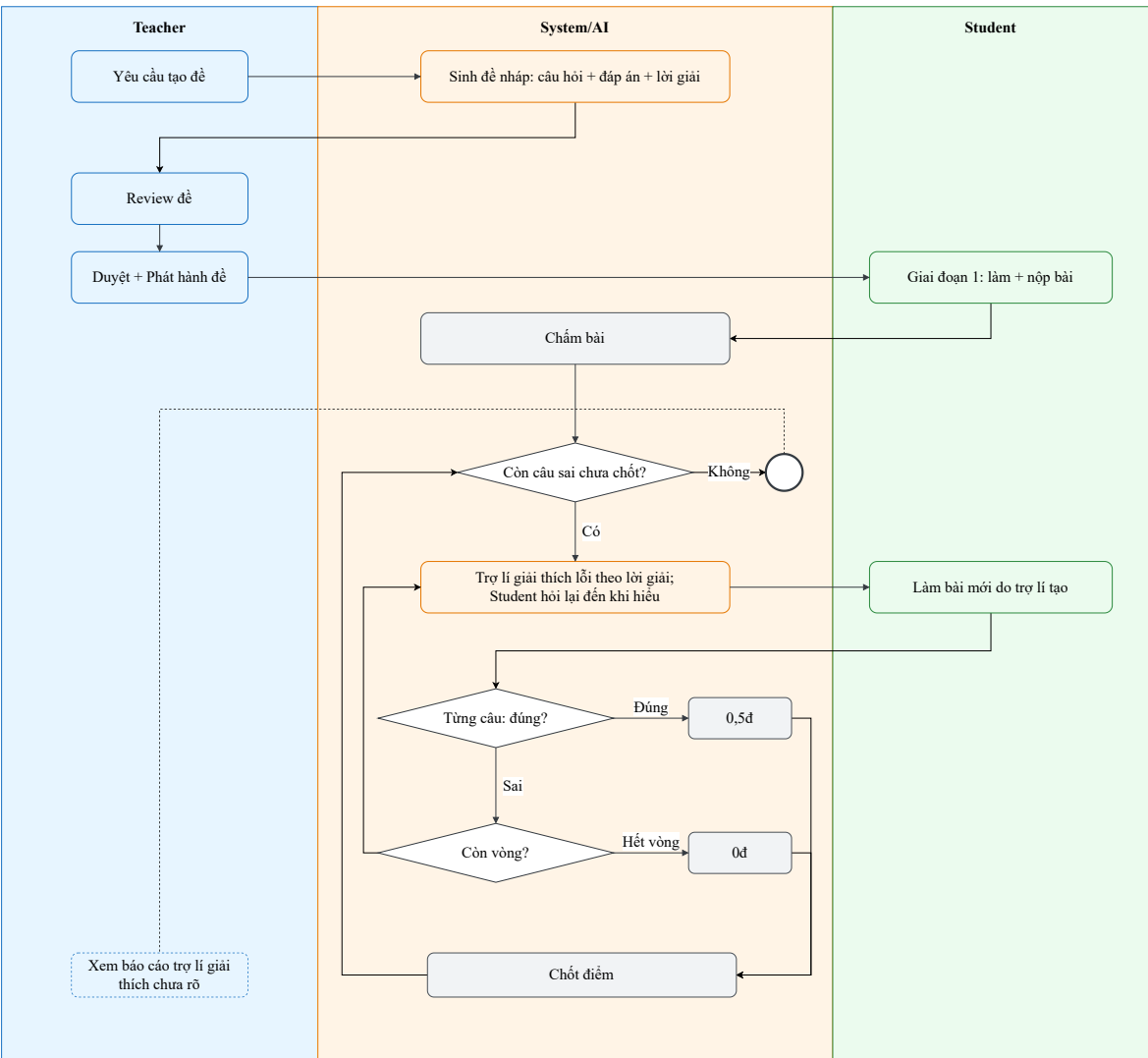
của giai đoạn 2 là một mốc chung cả lớp nhìn thấy suốt ngày — không cắt thì mốc ấy không tồn tại.

4.4.3 Giáo viên làm gì?

Học sinh báo cáo một câu hoặc một đoạn hội thoại là *giải thích chưa rõ*. Giáo viên xem được các báo cáo này và giải thích để đưa vào bài dạy trên lớp.

4.5 Business Workflows

Hình 4.1 cho thấy luồng tổng quan; hình 4.2 cho thấy trách nhiệm của từng vị trí.



Hình 4.2: Business Workflows

Chương 5

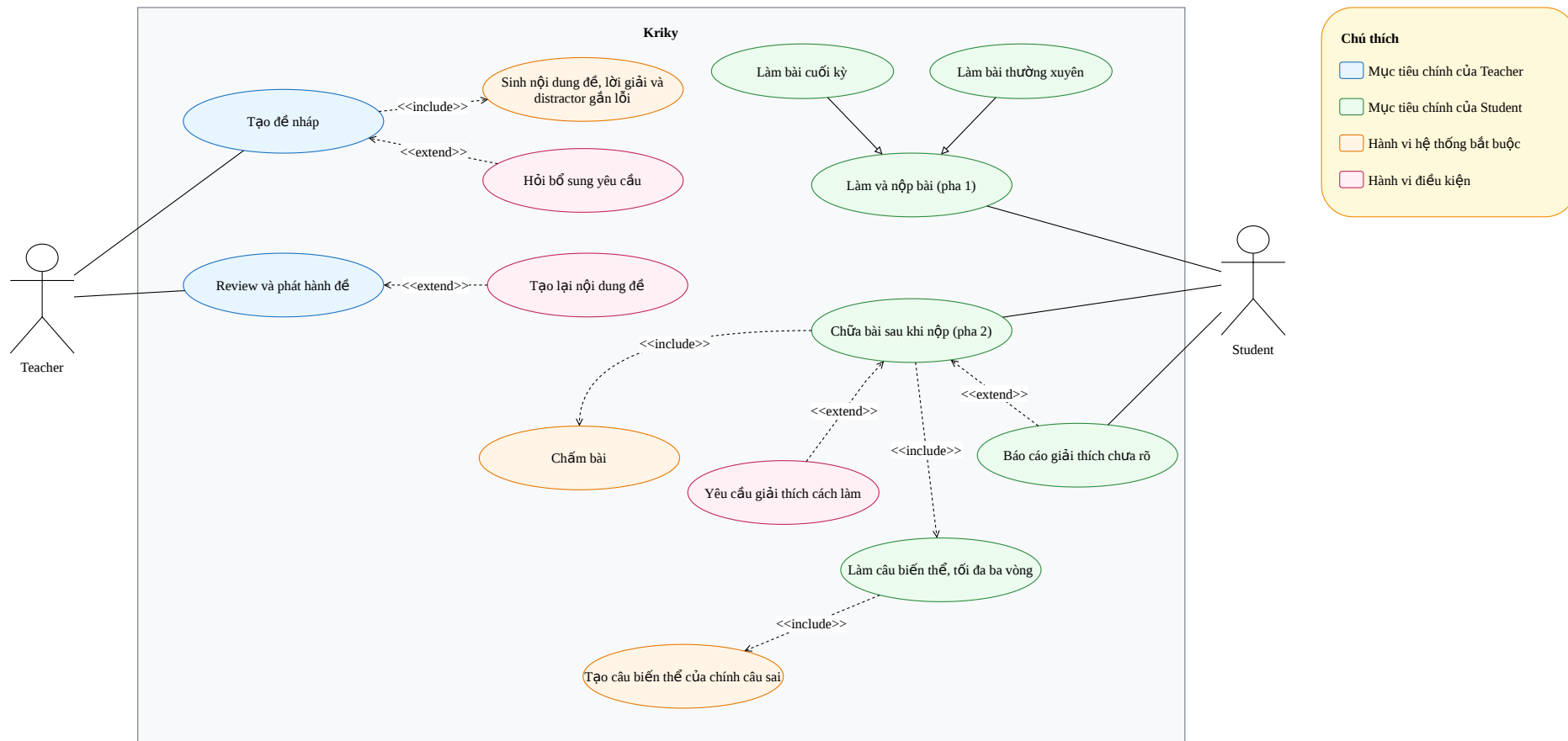
Use case

5.1 Sáu use case

Bảng 5.1: Sáu use case và actor chính của từng cái

Mã	Actor chính	Tên
UC-01	Teacher	Tạo đề nháp
UC-02	Teacher	Review và phát hành đề
UC-03	Student	Làm và nộp bài (giai đoạn 1)
UC-04	Student	Chữa bài sau khi nộp (giai đoạn 2)
UC-05	—	Hệ thống tạo câu biến thể
UC-06	Student	Báo cáo chỗ trợ lý giải thích chưa rõ

UC-05 không có actor trực tiếp: đó là hành vi hệ thống, được <<include>> từ UC-04.



Hình 5.1: Use Case Diagram

5.2 Đặc tả từng use case

5.2.1 UC-01 — Tạo đề nháp

Điều kiện trước. Teacher biết chủ đề hoặc mục tiêu học tập cần kiểm tra, và mô tả được yêu cầu ở mức đủ hiểu.

Luồng chính. Teacher nhập yêu cầu và các ràng buộc (chủ đề, số câu, độ khó, thời gian làm bài, loại bài). Hệ thống phân tích, sinh đề nháp gồm câu hỏi, phương án, đáp án đúng, lời giải và metadata, rồi hiển thị cho Teacher.

Quan hệ. *Tạo đề nháp <<include>> Sinh nội dung đề. Hỏi bổ sung yêu cầu <<extend>> Tạo đề nháp*, khi yêu cầu ban đầu chưa đủ rõ — đây chính là một trong hai nơi Giáo viên ra quyết định ở mục 2.6.

5.2.2 UC-02 — Review và phát hành đề

Điều kiện trước. Đã có đề nháp.

Luồng chính. Teacher xem từng câu hỏi, xem đáp án đúng, lời giải và phương án sai; chỉnh sửa nếu cần; duyệt đề; rồi phát hành với sáu tham số.

Quan hệ. *Tạo lại nội dung đề <<extend>> Review và phát hành đề*, khi Teacher thấy một phần hoặc toàn bộ đề chưa phù hợp.

5.2.3 UC-03 — Làm và nộp bài

Điều kiện trước. Bài đã được phát hành và Student có quyền truy cập.

Luồng chính. Student mở bài, đọc câu hỏi, chọn đáp án, nộp bài. Hệ thống ghi nhận, và chấm bài.

Quan hệ. *Làm bài thường xuyên và Làm bài cuối kỳ* đều generalize lên *Làm và nộp bài (giai đoạn 1)*.

Sơ đồ *không dùng <<include>>* từ *Làm và nộp bài* sang *Chấm bài*: chấm bài là bước xử lý *sau* khi có bài nộp, không phải hành vi bắt buộc để Student hoàn thành mục tiêu nộp bài của mình.

Kết quả. Bài nộp được ghi nhận và được chấm. **Nộp bài không kết thúc bài kiểm tra** — nó kết thúc giai đoạn 1.

5.2.4 UC-04 — Chữa bài

Điều kiện trước. Student đã nộp bài ở giai đoạn 1 và hệ thống đã chấm; chưa quá hạn kết thúc giai đoạn 2.

Luồng chính. Student mở kết quả; mở màn hỏi trợ lí, nơi mọi câu sai nằm cùng một màn và phần giải thích không tính giờ; bấm nút làm bài mới để khởi động đồng hồ của lượt; làm câu biến thể của từng câu sai. Câu đúng thì chốt 0,5; câu còn sai thì sang vòng tiếp, tối đa ba vòng, hết ba vòng thì chốt 0.

Ngoại lệ. Hết hạn giai đoạn 2 giữa một lượt thì lượt bị *dừng* và các câu còn dở nhận 0 điểm. Student không mở giai đoạn 2 lần nào thì mọi câu sai nhận 0 điểm.

Quan hệ. *Chữa bài* <<include>> *Chấm bài và tra cứu lỗi sai*, và <<include>> *Làm câu biến thể, tối đa ba vòng. Yêu cầu giải thích cách làm và Báo cáo giải thích chưa rõ* đều <<extend>> từ nó.

Quan hệ với *Làm câu biến thể* là <<include>> chứ không phải <<extend>>, **vì giai đoạn 2 bắt buộc**: nó không phải một nhánh có thể không xảy ra. Đây là ví dụ rõ nhất trong cả sơ đồ về việc một quyết định sự phạm đối luôn kiểu quan hệ trên hình.

5.2.5 UC-05 — Tạo câu biến thể

Điều kiện trước. Một câu ở giai đoạn 1 bị làm sai.

Luồng chính. Hệ thống lấy câu Student làm sai, sinh biến thể của *chính câu đó* — giữ nguyên cấu trúc và lỗi cần kiểm, thay dữ kiện — rồi đưa thẳng tới Student. Kết quả của câu biến thể quyết định câu gốc chốt 0,5 hay sang vòng tiếp.

5.2.6 UC-06 — Báo cáo Trợ lí giải thích chưa rõ

Điều kiện trước. Student đang ở giai đoạn 2, và trợ lí đã trả lời ít nhất một lần.

Luồng chính. Student bấm nút báo cáo “Trợ lí giải thích chưa rõ”. Hệ thống ghi lại **cả đoạn hội thoại của bài kiểm tra đó** kèm những câu Student làm sai để giáo viên có thể xem được.

Chương 6

Kiến trúc và mô hình dữ liệu

Chương này mô tả hệ thống được chia thành những phần nào, mỗi phần được phép quyết định gì, và dữ liệu nằm ở đâu. Ranh giới đáng chú ý ở đây không phải ranh giới kỹ thuật mà là ranh giới **quyền quyết định**: phần nào được phép kết luận, và phần nào chỉ được phép báo lại.

6.1 Năm thành phần, và cái gì làm nên một thành phần

Hệ thống chia làm năm thành phần, nhưng con số năm không phải điều đáng nói. Điều đáng nói là *luật* sinh ra nó — vì cùng một luật ấy sẽ trả lời câu hỏi thành phần thứ sáu nên trông như thế nào, còn một danh sách năm cái tên thì không.

Một phần việc tách ra thành một thành phần riêng khi nó khác ở **một trong ba trục**:

1. **Quyền quyết định** — nó được *kết luận*, hay chỉ được *báo lại*?
2. **Tầm với** — nó cầm chìa khoá của kho nào? Quyết định điều này là quyết định nó *có thể* làm hỏng những gì.
3. **Có ai đang chờ không** — có người đứng đợi kết quả không? Có người chờ thì việc ấy là một lời gọi và phải trả lời ngay. Không ai chờ thì nó *không thể* là một lời gọi: giáo viên đã rời màn hình từ lâu, nên không còn màn hình nào để báo lỗi về.

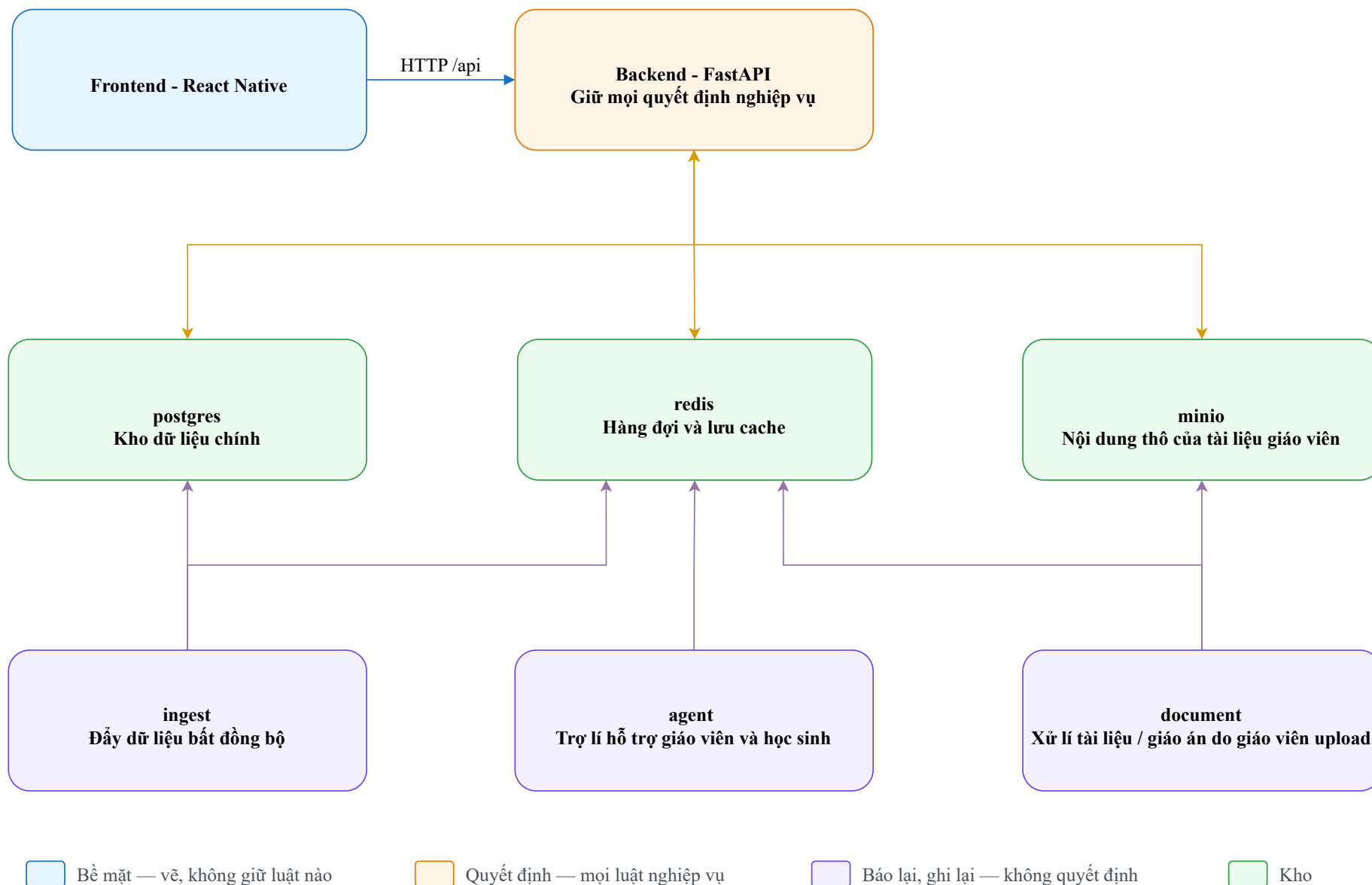
Năm thành phần rơi ra từ ba trục đó, chứ không phải từ việc chia theo danh từ nghiệp vụ.

Ba thành phần cuối bảng không có ai chờ chúng, nên cả ba **không mở một bề mặt nào cho ai gọi vào**: việc chỉ tới được chúng qua hàng đợi. Đó là một tính chất của thiết kế, không phải một chi tiết cài đặt — nó là thứ khiến một lần xử lý chậm không giữ một kết nối nào treo suốt thời gian chờ.

Hình 6.1 vẽ năm thành phần ấy cùng ba kho và hai package dùng chung, sắp theo đúng ba trục trên.

Bảng 6.1: Năm thành phần, đọc theo ba trục

Thành phần	Vai trò	Quyết định	Chìa khoá
fe	Giao diện cho giáo viên và học sinh. Chỉ nói chuyện với be	Không	Không có
be	Nơi giữ mọi quyết định nghiệp vụ, và là cửa vào duy nhất của hệ thống	Mọi quyết định	Kho dữ liệu chính, kho tệp
agent	Soạn nội dung bằng AI: câu hỏi của đề, câu biến thể, lượt trả lời trong hội thoại. Nộp nội dung và bằng chứng	Không	Không có cái nào
document	Đọc tài liệu giáo viên tải lên: có chữ hay không, bao nhiêu trang. Báo lại, không sở hữu dữ liệu nào	Không	Kho tệp, và chỉ đọc
ingest	Ghi lại thứ document báo về	Không	Kho dữ liệu chính, để ghi



Hình 6.1: System Architecture — năm thành phần đọc theo quyền quyết định và chia khoá, ba kho, hai package dùng chung

Giao diện không biết agent tồn tại. Mọi thứ nó cần đều đi qua be, nên hệ thống chỉ có **một cửa ra vào**. Thêm một cửa thứ hai là thêm một chỗ phải tự kiểm lại mọi luật nghiệp vụ, và hai chỗ kiểm thì sớm muộn khác nhau ở một luật.

6.2 Ranh giới quyền quyết định

Đây là ràng buộc quan trọng nhất của kiến trúc, và nó là một ràng buộc *ng nghiệp vụ* chứ không phải một lựa chọn kỹ thuật. Luật chỉ có một câu: **agent nộp nội dung, be quyết định nội dung ấy có được dùng hay không**. Không có chỗ nào trong hệ thống để một mô hình tự đưa thứ gì vào đề, vào điểm, hay vào bài của một học sinh.

Chỗ thấy rõ nhất là việc soạn đề:

- agent trả về một **đề xuất**: câu hỏi, các phương án, ánh xạ từ mỗi phương án nhiều sang một lỗi cụ thể, và lời giải.
- be **kiểm lại từng câu trước khi ghi** — đúng một đáp án đúng, mọi phương án nhiều đều gắn một lỗi, ít nhất hai cách giải — rồi mới lưu. Và một đề chỉ rời trạng thái nhấp khi giáo viên bấm duyệt.

Ba luật ấy là luật nghiệp vụ, không phải chi tiết kỹ thuật; chỗ đáng nói ở đây là *nơi* chúng được canh. Một luật chỉ nằm trong lời dặn gửi cho mô hình là một luật không ai canh: mô hình bỏ qua một câu dặn được, còn một phép kiểm trước khi ghi thì không.

Việc chấm đi theo đúng ranh giới ấy, và đi xa thêm một bước: agent **không bao giờ** trả về điểm. Đáp án đúng nằm trong kho dữ liệu chính, mà agent thì không giữ chìa khoá của kho ấy — nên nó không phán được đúng sai kể cả khi muốn. Workflow 3 ở chương 4 nói vì sao chỗ đặt ấy là có ý.

Nếu để agent tự quyết, công giáo viên sẽ nằm *bên trong* phần AI — và một công nằm trong thứ nó phải canh thì không còn là công.

6.3 Ranh giới chìa khoá: ai chạm được kho nào

Hệ thống có ba kho — kho dữ liệu chính, hàng đợi việc, và kho tệp — và với mỗi kho, câu đáng hỏi là ai được giữ chìa khoá.

Kho dữ liệu chính thuộc về be và ingest. Nó giữ lớp, học sinh, đề, lần làm bài, số điểm ba mức, bộ đếm vòng, từng lượt làm lại kèm đề đã sinh ra, các đoạn hội thoại, và các báo cáo. Lý do nó tồn tại là một luật nghiệp vụ: hạn giai đoạn 2 do giáo viên đặt, tính bằng giờ hoặc ngày, nên trạng thái bài làm không sống được ở một chỗ chỉ giữ dữ liệu trong một giờ.

agent và document không có chìa khoá của kho dữ liệu chính. Hệ quả trực tiếp là **mỗi việc giao cho chúng phải tự chứa đủ thông tin**: việc giải thích một lượt mang theo cả câu hỏi, phương án, lời giải và lỗi đã soạn, chứ không mang một mã số để đi tra. Ràng buộc này nghe như thêm việc, nhưng nó đổi lại được một thứ đáng giá: hai thành phần ấy không thể vô tình đọc hay sửa thứ không thuộc phần việc của mình. Chúng không có đường nào để đi tới đó.

ingest thì *có* chìa khoá, và đó không phải một ngoại lệ cho tiện. Điều làm agent với document thành hai thành phần chỉ-đọc-và-báo là chúng **không ghi vào dữ liệu nào**, chứ không phải

việc chúng chạy riêng. ingest có đúng một việc là ghi lại kết quả đọc tài liệu, nên nó cần đường tới chỗ ghi — và nó không quyết định gì cả.

Kho tệp giữ byte của tài liệu, tức nội dung thô của tệp. Đây là kho duy nhất không phải cơ sở dữ liệu, và là kho duy nhất mà một thành phần chỉ đọc chứ không bao giờ ghi: be ghi vào, document đọc ra, và không có chiều ngược lại. Trong bảng dữ liệu chỉ có một khoá trở tới tệp, không có một chữ nào của nội dung.

Việc document giữ chìa khoá kho tệp không phá ranh giới ở trên, và lý do đáng nói ra chứ không nên để người đọc tự suy: kho tệp là nơi **byte** nằm, không phải nơi **sự thật** nằm. Một tài liệu đã xử lý xong hay chưa thì chỉ bảng dữ liệu biết, và document không đọc được bảng đó. Nó đọc một tệp rồi báo lại; nó không bao giờ biết câu trả lời của mình đã được ghi hay chưa.

6.4 Các thành phần nói chuyện với nhau thế nào

Có hai cơ chế chờ việc, và một tín hiệu rất ngắn không chờ gì.

Một lời gọi, khi có người đang chờ. Giao diện gọi be. Đó là đường duy nhất đi vào hệ thống từ bên ngoài, và cũng là lời gọi duy nhất giữa hai thành phần.

Một việc trên hàng đợi, khi không ai chờ. Mọi trao đổi còn lại đi đường này. Bên gửi đặt một việc lên hàng đợi kèm **tên của việc** — một chuỗi ký tự — rồi đi tiếp. Bên nhận lấy việc xuống và làm.

Chỗ đáng nói là *cái chuỗi ấy chính là ranh giới*. Hai thành phần thông nhất với nhau trên một chuỗi, nên **không bên nào phải biết bên kia tồn tại dưới dạng mã nguồn**. Nếu thay vì vậy bên gửi gọi thẳng một hàm của bên nhận, thì hai thành phần dính vào nhau: đổi một cái là phải đổi cái kia, và cái gọi là ranh giới chỉ còn là một thư mục khác tên.

Hệ quả thứ hai của việc không ai import ai: **payload phải tự chứa đủ**. Bên nhận không tra cứu được gì — nó không có chìa khoá — nên mọi thứ nó cần phải nằm sẵn trong việc được giao. Đây chính là ràng buộc ở mục trên, nhìn từ phía đường truyền.

Và một tín hiệu rất ngắn, khi có thứ vừa đổi. Thư viện tài liệu của giáo viên đổi *sau* lúc tải lên, mà không lời gọi nào của màn hình gây ra việc ấy. Nên khi ingest ghi xong một kết quả, nó phát một tín hiệu; be nghe được thì đẩy xuống màn hình. Tín hiệu ấy không chờ nội dung, và mục 6.6 nói vì sao nó cố ý nghèo như vậy.

6.5 Thứ dùng chung, và cái giá đã nhận

Không thành phần nào dùng mã nguồn của thành phần khác. Nhưng có hai thứ *phải* dùng chung, và cả hai đều là **khai báo**, không phải hành vi.

Bảng từ vựng của những việc đi qua hàng đợi. Tên của từng loại việc, hình dạng của từng thông điệp, và các tập giá trị hợp lệ. Nó phải dùng chung vì hai bên đọc cùng một việc; nếu mỗi bên tự khai một bản, hai bản sẽ lệch nhau vào một ngày không ai để ý. Nó chứa *chỉ*

dữ liệu — không một luật nghiệp vụ nào, vì một luật đặt ở đây sẽ thành luật mà không thành phần nào chịu trách nhiệm.

Khai báo các bảng dữ liệu. be và ingest cùng ghi vào kho dữ liệu chính, nên hai thành phần ấy phải nói đúng một định nghĩa bảng.

Cái giá của việc này nên nói thẳng chứ không nên giấu: hai thành phần dùng chung một định nghĩa schema thì **đổi một cột là một thay đổi phá vỡ cho cả hai cùng lúc**, kể cả khi cột ấy nằm trong một bảng mà một bên không bao giờ đụng tới. Đổi lại là một ranh giới *kiểm được bằng máy* thay vì một ranh giới chỉ viết trong tài liệu — xem mục 6.8. Đây là một đánh đổi đã cân nhắc, không phải một chỗ sơ ý.

6.6 Việc gì đi qua hàng đợi, việc gì không

Mọi việc cần gọi mô hình đều chạy qua một hàng đợi, không chờ ngay trong lúc người dùng còn đứng đó. Một lần gọi mô hình lâu tới mức giữ một kết nối mở suốt thời gian chờ là không hợp lý.

Bảng 6.2: Bốn việc thật sự cần gọi mô hình

Việc	Khi nào xảy ra
Viết một câu hỏi	Mỗi câu của bộ đề giáo viên yêu cầu. Một câu một việc , không phải cả bộ một việc
Sinh câu biến thể	Ngay khi học sinh nộp bài giai đoạn 1, đề câu có sẵn lúc em bấm nút
Trả lời một lượt hội thoại	Mỗi lượt học sinh hỏi lại ở giai đoạn 2
Đề xuất bước tiếp theo	Mỗi bước của một lượt trò chuyện với giáo viên

Việc viết câu hỏi tách thành *một câu một việc* là một quyết định có con số đứng sau, không phải chuyện thích hay không thích. Hệ thống có một hạn chờ cho *một* việc. Gộp cả bộ đề vào một việc thì một đề năm mươi câu tiêu gấp năm mươi lần cái hạn ấy, mà phép kiểm hạn chờ vẫn báo xanh suốt thời gian đó — vì nó đang kiểm sai thứ.

Chấm bài không đi qua hàng đợi. Với trắc nghiệm, chấm là một phép so giữa phương án đã chọn và đáp án đúng đã lưu sẵn, nên nó chạy ngay trong lúc học sinh nộp bài. Hệ quả nhìn thấy được trên màn hình: giữa lúc nộp và lúc xem kết quả **không có trạng thái đang chấm nào**.

Màn hình được báo, thay vì phải hỏi lại. Có những chỗ dữ liệu hiện ra dần, hoặc đổi sau khi người dùng đã rời đi. Ở những chỗ ấy hệ thống tự đẩy xuống. Có ba chỗ như vậy:

- **Hội thoại ở giai đoạn 2** hiện chữ dần cho học sinh.
- **Hội thoại của giáo viên** đẩy từng bước của một lượt ngay khi bước đó xảy ra.
- **Thư viện tài liệu của giáo viên** đổi *sau* lúc tải lên — một tài liệu đi từ *đang xử lý* sang *sẵn sàng* vài giây sau, đôi khi lâu hơn.

Hai kênh đầu là *kênh tăng tốc cảm giác, không phải nguồn sự thật*. Nội dung được lưu xong rồi mới phát ra, nên mất kết nối chỉ mất phần hoạt hình.

Kênh thứ ba thì khác, và khác có chủ đích: nó chỉ báo *có gì đó vừa đổi*, rồi màn hình tự đọc lại danh sách. Một thông báo chờ sẵn nội dung mà bị mất giữa đường sẽ để lại một màn hình sai vĩnh viễn. Một thông báo chỉ nói có gì đó đổi thì lần báo sau sửa luôn lần trước.

6.7 Thành phần nào ngừng thì mất gì

Một kiến trúc chia nhiều phần chỉ đáng giá nếu một phần hỏng không kéo cả hệ thống theo. Bảng dưới nói rõ từng trường hợp, và vài dòng trong đó là *lựa chọn* chứ không phải hệ quả tự nhiên.

Bảng 6.3: Mỗi thành phần ngừng thì mất gì, còn gì

Ngừng	Mất	Còn
Hàng đợi	Mọi việc cần gọi mô hình. Tải tài liệu lên vẫn cất được tệp nhưng hỏng ngay , và hàng nói ra điều đó	Làm bài, nộp bài, xem điểm giai đoạn 1
Kho tệp	be không khởi động — cố ý	—
Kho dữ liệu chính	be không khởi động	—
agent	Soạn đề, hội thoại	Mọi thứ còn lại
document	Tài liệu mới đứng ở <i>đang xử lý</i> , rồi đọc ra <i>xử lý hỏng</i>	Thư viện vẫn mở, tài liệu cũ vẫn dùng được
ingest	Kết quả đọc tệp không vào được kho. Trên màn hình nhìn <i>giống hết</i> dòng trên	Nhưng việc không mất : nó nằm lại trên hàng đợi, bật lại thì chạy tiếp

Hai dòng đầu tương phản nhau, và sự tương phản ấy là có chủ ý. Hàng đợi chết thì be vẫn khởi động và chỉ những đường cần mô hình mới báo lỗi — vì làm bài, nộp bài và xem điểm giai đoạn 1 không cần tới mô hình, nên không có lý do gì để chúng chết theo. Kho tệp chết thì be *dừng ngay lúc khởi động*, và đó không phải một thiếu sót: một đường mà việc duy nhất của nó là cất tệp thì không có phiên bản suy giảm nào — nó hoặc cất được, hoặc nói dối.

Hai dòng cuối cần đọc kỹ, vì chúng *trông* giống nhau trên màn hình mà khác nhau ở chỗ quan trọng nhất. Hệ thống không ghi trạng thái *xử lý hỏng* vào đâu cả — nó **suy ra lúc đọc**, khi một tài liệu nằm ở *đang xử lý* lâu hơn ngưỡng cho phép. Nên dù document chết hay ingest chết, quá ngưỡng ấy là giáo viên đọc ra cùng một câu. Cái khác nằm bên dưới: document chết thì chưa ai đọc tệp, còn ingest chết thì tệp đã đọc xong và câu trả lời đang **nằm chờ** trên hàng đợi.

Dòng ingest là dòng duy nhất *đã* được kiểm bằng tay — năm dòng kia suy từ mã nguồn. Phép kiểm: tắt thành phần ấy rồi tải một tài liệu lên, tài liệu đứng ở *đang xử lý*; bật lại, việc vẫn được làm nốt và tài liệu chuyển sang *sẵn sàng*. Đó là bằng chứng việc đã nằm an toàn trên hàng đợi chứ không biến mất cùng thành phần đọc nó.

6.8 Cách giữ cho ranh giới không mục

Mọi ranh giới trong chương này đều là một câu văn, và một câu văn thì không tự giữ được. Người viết mã sau sáu tháng không đọc lại báo cáo; họ đọc mã. Nên mỗi ranh giới đáng giữ đều được gắn một phép kiểm tự động, chạy mỗi lần mã nguồn thay đổi.

Bảng 6.4: Ranh giới, và thứ bắt được khi nó bị phá

Ranh giới	Bắt được bằng
Không thành phần nào dùng mã nguồn của thành phần khác	Kiểm quan hệ phụ thuộc, chạy trước mỗi lần ghi nhận thay đổi
Hai package dùng chung không được biết thành phần nào tồn tại	Cùng phép kiểm trên
agent và document không giữ chìa khoá kho dữ liệu	Phép kiểm ở tầng dự án
Chỉ một chỗ được phép dựng bảng; mọi chỗ khác chỉ được đối chiếu	Phép kiểm ở tầng dự án
Hai đầu của tín hiệu đọc cùng một tên kênh từ bảng từ vựng chung	Phép kiểm ở tầng dự án
agent không phát ra quyết định định tuyến	Kiểm thử tự động

Hiện có hai mươi phép kiểm ở tầng dự án cùng ba hợp đồng phụ thuộc. Cách duy nhất biết một phép kiểm còn sống là *thử phá*: cố tình viết sai một dòng, xác nhận phép kiểm báo đỏ, rồi hoàn tác. Một phép kiểm chưa bao giờ đỏ là một phép kiểm chưa ai chứng minh là có tác dụng.

6.9 Kiến trúc không nói nơi chạy

Chương này không nói thành phần nào chạy ở đâu, và đó là một lựa chọn. Nơi chạy sẽ đổi — từ một máy cá nhân sang một máy chủ, rồi có thể sang một cụm điều phối — còn ranh giới quyền quyết định thì không. Một sơ đồ trộn hai thứ ấy vào nhau sẽ sai mỗi lần hạ tầng đổi, mà phần sai lại là phần không đáng vẽ.

Thiết kế hiện tại mang đi được, vì ba tính chất. Không thành phần nào viết cứng cấu hình vào mã nguồn: mọi giá trị đều đọc được từ biến môi trường, và một tệp cấu hình ở gốc dự án chỉ đóng vai trò giá trị mặc định cho máy phát triển. Không thành phần nào giữ trạng thái của riêng nó trên đĩa — dữ liệu ở kho dữ liệu, tệp ở kho tệp. Và mọi trao đổi đều qua hai cơ chế ở mục 6.4, không qua bộ nhớ chung. Hệ quả: thêm một bản sao của một thành phần đọc hàng đợi chỉ là thêm một người tiêu thụ, không cần ai phối hợp với ai. Tín hiệu báo thư viện đổi cũng phát theo kiểu quảng bá, nên nhiều bản sao của be đều nghe được — một giáo viên nói vào bản sao nào cũng được báo.

Ba chỗ chưa sẵn sàng, nói thẳng ra. Thứ nhất, việc dựng bảng dữ liệu hiện nằm trong lúc be khởi động. Một bản sao thì đúng; nhiều bản sao khởi động cùng lúc là nhiều tiến trình cùng dựng một schema, và cùng gieo dữ liệu mẫu. Việc ấy sẽ phải tách ra chạy trước, một

lần, chứ không nằm trong đường khởi động của một thành phần phục vụ. Thứ hai, hệ thống có một đường để hỏi *tiến trình này còn sống không* nhưng chưa có đường hỏi *nó đã sẵn sàng nhận việc chưa*. Đường đang có cố ý *không* thăm dò các kho — nhờ vậy nó phân biệt được một tiến trình chết với một kho chết — và đó là một quyết định nên giữ, nhưng nó không thay được đường thứ hai. Thứ ba, hai cửa sổ trình duyệt cùng chạy một lượt trò chuyện của giáo viên thì không có khoá nào ngăn; một bản sao đã vậy, nhiều bản sao thì nặng hơn.

Ba chỗ này đều là việc của bước triển khai, không phải lỗi của thiết kế hiện tại. Nếu ra ở đây để chúng là những việc đã biết tên, chứ không phải những thứ phát hiện ra vào ngày đây lên máy chủ.

6.10 Mô hình dữ liệu

Mô hình dữ liệu có hai mươi một bảng, chia theo năm nhóm. Mỗi bảng trả lời đúng một câu hỏi nghiệp vụ, và phần ghi chú dưới đây nói vì sao ranh giới giữa chúng nằm ở chỗ đó.

Bảng 6.5: Hai mươi một bảng, theo năm nhóm

Bảng	Giữ gì
Nhóm 1 — người và lớp	
classes	Một lớp giáo viên tạo. Lớp thuộc về giáo viên, không thuộc về nhà trường. Tên lớp <i>không</i> phải định danh
students	Họ tên, mã học sinh, thuộc lớp nào. Mã học sinh là khoá con người dùng
teachers	Họ tên và mã giáo viên
Nhóm 2 — nội dung đề	
assessments	Tác giả, tiêu đề, môn, khối, và <i>trạng thái</i> — bốn giá trị của vòng đời một đề: rỗng, có câu hỏi, đã duyệt, đã phát hành
questions	Đề bài, thứ tự câu học sinh nhìn thấy, mục tiêu học tập
options	Nhãn, nội dung, dấu đáp án đúng, và <i>lỗi mà phương án này đại diện</i> . Ô lỗi để trống ở đúng một dòng mỗi câu, và dòng đó chính là đáp án đúng
methods	Các cách giải của một câu. Bắt buộc nhiều hơn một
publications	Sáu tham số phát hành, một dòng cho mỗi cặp đề và lớp, nên thu hồi được từng lớp một
documents	Tài liệu giáo viên tải lên: tên tệp, kích thước, khoá trở tới kho tệp, trạng thái xử lý, số trang, lý do hỏng. Không giữ byte nào. Thuộc về giáo viên, không thuộc về đề
draft_briefs	Yêu cầu soạn của một đề: môn, khối, phạm vi, tổng số câu đã xin
draft_items	Một ô cho mỗi câu đang soạn, kèm trạng thái, số lần đã thử, và <i>lý do</i> lần gần nhất bị loại
Nhóm 3 — bài làm và điểm	

tiếp trang sau

Bảng	Giữ gì
attempts	Một học sinh làm một đề: lớp lúc bắt đầu, giờ bắt đầu, giờ phải xong, giờ đã nộp
answers	Lựa chọn ở giai đoạn 1, ghi mỗi lần bấm chữ không gom tới lúc nộp
question_outcomes	Số điểm: mức điểm cuối, lý do, số vòng đã dùng, đã chốt hay chưa
Nhóm 4 — giai đoạn 2	
rounds	Một lượt làm lại và giờ kết thúc của nó, đã cắt xuống hạn giai đoạn 2 nếu vượt
round_items	Câu sinh cho lượt đó, kèm phương án và lời giải — giữ cả đáp án đúng ở đây là thứ cho phép hệ thống tự chấm lượt
pregenerated_items	Câu viết <i>trước</i> khi học sinh bấm nút, trong lúc em còn đang đọc giải thích
chat_messages	Từng lượt hội thoại, lưu <i>trước</i> khi phát ra màn hình
reports	Báo cáo <i>giải thích chưa rõ</i> , gắn với cả bài chứ không với một tin nhắn
Nhóm 5 — hội thoại của giáo viên	
teacher_conversations	Một mạch hội thoại của một giáo viên. Tiêu đề do trợ lý đặt sau lượt đầu, giáo viên sửa được; xóa là <i>xoá mềm</i>
teacher_turns	Từng <i>bước</i> của một lượt: lời nói, hoặc công cụ đã chạy, kèm chủ thể mà bước đó nói về

6.10.1 Ba chỗ đáng giải thích

Số điểm tách khỏi câu trả lời. Một câu có một câu trả lời ở giai đoạn 1 và nhiều lượt ở giai đoạn 2, nhưng chỉ có **một** kết luận. Gộp điểm vào bảng câu trả lời sẽ buộc phải trả lời câu hỏi *câu trả lời nào mang điểm*, mà câu hỏi ấy không có đáp án đúng. Mức điểm trong sổ chỉ đi lên, không bao giờ đi xuống — đó là cách luật *giai đoạn 2 chỉ cộng* ở mục 2.5 được bảo đảm bằng dữ liệu chứ không bằng lời hứa.

Câu của một lượt không nằm chung với câu của đề. Một câu biến thể thuộc về đúng một lượt và không bao giờ bị tìm kiếm xuyên qua các lượt khác. Đưa nó vào bảng câu hỏi của đề thì mọi chỗ đọc đề đã duyệt đều phải nhớ bỏ qua loại câu hỏi thứ hai này. Chỉ cần một chỗ quên là đề của giáo viên có thêm những câu họ chưa từng duyệt.

Khoá nội dung là một trạng thái của đề, không phải một cờ trên từng câu. Duyệt là thao tác trên cả đề, nên thứ ghi lại nó cũng phải nằm ở cấp đề. Một cờ trên từng câu sẽ cho phép một đề nửa khoá nửa mở, mà trạng thái đó không có nghĩa nghiệp vụ nào.

6.11 Vì sao chưa vẽ sơ đồ lớp hay sơ đồ thực thể

Báo cáo này mô tả dữ liệu bằng bảng chứ không bằng sơ đồ thực thể, và đó là một lựa chọn chứ không phải thiếu sót.

Một sơ đồ thực thể nói về *quan hệ* giữa các bảng. Mà quan hệ ở đây gần như toàn là quan hệ cha con hiển nhiên: lớp có học sinh, đề có câu hỏi, câu hỏi có phương án. Một sơ đồ như vậy tốn một trang để nói lại thứ bảng đã nói.

Thứ *thật sự* khó hiểu trong mô hình này không phải quan hệ, mà là ba chỗ vừa giải thích ở trên. Vì sao điểm tách khỏi câu trả lời, vì sao câu của một lượt nằm riêng, và vì sao khoá nội dung là trạng thái của đề. Cả ba đều là lý do, và một sơ đồ thì không vẽ được lý do.

Sơ đồ ấy sẽ đáng vẽ khi mô hình có một quan hệ mà người đọc không đoán ra được từ tên bảng.

Tài liệu tham khảo

Báo cáo này mô tả một hệ thống đang chạy được, nên nguồn của nó là chính mã nguồn và các bản ghi quyết định đi kèm, chứ không phải tài liệu bên ngoài. Mọi con số và mọi luật nêu trong báo cáo đều có một chỗ tương ứng trong kho dưới đây.

Mã nguồn và tài liệu kỹ thuật của dự án

<https://github.com/biabeogo147/AI-Assessment-and-Feedback-Assistant>

Kho chứa năm thành phần đã mô tả ở chương 6, cùng ba nhóm tài liệu mà báo cáo này rút gọn từ đó:

- docs/decisions/ — các bản ghi quyết định nghiệp vụ. Mỗi quyết định ở đó ghi cả phần *đã cân nhắc những gì và nơi nào thi hành nó*, hai phần mà báo cáo này không chở.
- docs/overview/ — mô tả nghiệp vụ, kiến trúc và mô hình dữ liệu. Nguồn của chương 4, 5 và 6.
- docs/diagrams/ — tệp gốc của mọi sơ đồ trong báo cáo. Các hình ở đây được *xuất ra* từ chúng chứ không vẽ lại, và một phép kiểm tự động trong kho sẽ báo đỏ nếu một sơ đồ đổi mà hình chưa xuất lại.

Các luật được canh bằng máy

tools/check_contract.py và AGENTS.md trong kho trên.

Mục 6.8 nói rằng mỗi ranh giới đáng giữ đều có một phép kiểm tự động đứng sau. Hai tệp này là nơi những phép kiểm ấy sống, và là chỗ kiểm chứng được các con số nêu ở mục đó.